
Especificación de Requisitos de Software

Proyecto: Sistema de Control de Almacenes para FAME S.A. "SCAFAME"



Octubre 2025

Ficha del documento

Fecha	Revisión	Autor	Verificado dep. calidad.
11/11/2025	1.0	Miguel Gutiérrez	[Firma o sello]

Documento validado por las partes en fecha: [Fecha]

Por el cliente	Por la empresa suministradora
Fdo. D./ Dña [Nombre]	Fdo. D./Dña [Nombre]

Contenido

FICHA DEL DOCUMENTO	3
CONTENIDO	4
1 INTRODUCCIÓN	6
1.1 Propósito	6
1.2 Alcance	6
1.3 Personal involucrado	6
1.4 Definiciones, acrónimos y abreviaturas	6
1.5 Referencias	6
1.6 Resumen	6
2 DESCRIPCIÓN GENERAL	7
2.1 Perspectiva del producto	7
2.2 Funcionalidad del producto	7
2.3 Características de los usuarios	7
2.4 Restricciones	7
2.5 Suposiciones y dependencias	7
2.6 Evolución previsible del sistema	7
3 REQUISITOS ESPECÍFICOS	7
3.1 Requisitos comunes de los interfaces	8
3.1.1 Interfaces de usuario	8
3.1.2 Interfaces de hardware	8
3.1.3 Interfaces de software	8
3.1.4 Interfaces de comunicación	8
3.2 Requisitos funcionales	8
3.2.1 Requisito funcional 1	9
3.2.2 Requisito funcional 2	9
3.2.3 Requisito funcional 3	9
3.2.4 Requisito funcional n	9
3.3 Requisitos no funcionales	9
3.3.1 Requisitos de rendimiento	9
3.3.2 Seguridad	9
3.3.3 Fiabilidad	9
3.3.4 Disponibilidad	9
3.3.5 Mantenibilidad	10

3.3.6	Portabilidad	10
3.4	Otros requisitos	10
4	APÉNDICES	10

1 Introducción

1.1 Propósito

El presente documento tiene como propósito definir los requisitos funcionales y no funcionales del sistema SCAFAME (Sistema de Control y Administración de Inventarios para FAME S.A.), desarrollado para optimizar la gestión, control y trazabilidad del inventario institucional. Este documento servirá como referencia para el equipo de desarrollo, analistas, usuarios y auditores durante el ciclo de vida del sistema.

1.2 Alcance

SCAFAME es una aplicación web desarrollada para la Fábrica de Municiones del Ejército (FAME S.A.), que permite registrar productos, categorías, unidades de medida, movimientos de entrada y salida, y generar reportes en tiempo real. El sistema mejora la eficiencia y precisión en la administración de inventarios mediante interfaces intuitivas y automatización de procesos.

1.3 Personal involucrado

Nombre	Ing. Henry Gamarra
Rol	Líder de Proyecto
Categoría profesional	Ingeniero
Responsabilidades	Supervisión general y validación del sistema.
Información de contacto	hgamarra@fame.ec
Aprobación	Aprobado

Nombre	Miguel Gutiérrez
Rol	Desarrollador
Categoría profesional	Estudiante
Responsabilidades	Diseño, desarrollo e implementación del sistema.
Información de contacto	Magutierrez9@espe.edu.ec
Aprobación	Aprobado

1.4 Definiciones, acrónimos y abreviaturas

SCAFAME: Sistema de Control y Administración de Inventarios de FAME S.A.

FAME S.A.: Fábrica de Municiones del Ejército del Ecuador.

Inventario: Conjunto de materiales y productos almacenados.

Administrador: Usuario encargado de la gestión global del sistema.

Bodeguero: Usuario responsable del registro de movimientos.

1.5 Referencias

Referencia	Título	Ruta	Fecha	Autor
IEEE 830-1998	[Título] Standard for Software Requirements Specification.	https://standards.ieee.org/ieee/830/1222/	n.d.	IEEE
ISO/IEC/IEEE 29148:2018	Requisitos de Ingeniería de Software.	https://standards.ieee.org/ieee/830/1222/	n.d.	IEEE
FAME S.A.	Documentación interna de procesos de FAME S.A.		n.d.	FAME S.A.

1.6 Resumen

Este documento describe de forma detallada el propósito, alcance, contexto, requisitos funcionales y no funcionales del sistema SCAFAME. Además, define las restricciones técnicas, suposiciones, dependencias y la evolución prevista del sistema.

2 Descripción general

2.1 Perspectiva del producto

SCAFAME es un sistema web independiente desarrollado con tecnologías modernas (Angular, NestJS, PostgreSQL, Docker). Permite la gestión centralizada del inventario institucional de FAME S.A., integrando funcionalidades de registro, control y generación de reportes.

2.2 Funcionalidad del producto

El sistema incluye módulos de administración de usuarios, productos, categorías, unidades de medida, movimientos de inventario (ingresos y salidas), y generación de reportes. Cada módulo cuenta con validaciones y controles para garantizar la integridad de los datos.

2.3 Características de los usuarios

Tipo de usuario	Administrador
Formación	Profesional
Habilidades	Administración y Control
Actividades	Gestionar Inventario. Usuarios, Ingresos, Retiros y Reportes

Tipo de usuario	Bodeguero
Formación	Profesional
Habilidades	N/A
Actividades	Gestionar Inventario, Ingresos y Retiros

Tipo de usuario	Usuario
Formación	Profesional
Habilidades	N/A
Actividades	Ver Inventario, Solicitar Retiro

2.4 Restricciones

El sistema se desarrollará bajo el marco de tecnologías: Angular, NestJS, PostgreSQL y Docker. El acceso requiere conexión a red interna o VPN de FAME S.A. Las credenciales se gestionan mediante JWT.

2.5 Suposiciones y dependencias

Se asume que los usuarios contarán con credenciales válidas y conexión estable. El sistema depende de la disponibilidad del servidor Docker y la base de datos PostgreSQL.

2.6 Evolución previsible del sistema

Futuras versiones de SCAFAME podrían incluir integración con sistemas ERP, notificaciones automáticas y módulos de análisis de inventario predictivo.

3 Requisitos específicos

3.1 Requisitos comunes de los interfaces

El sistema SCAFAME gestionará la interacción entre el usuario, el hardware, el software y las comunicaciones a través de interfaces claramente definidas.

Todos los módulos deberán mantener coherencia visual, validaciones de datos y una comunicación eficiente con los servicios y base de datos.

3.1.1 Interfaces de usuario

El sistema contará con una interfaz web responsiva e intuitiva, desarrollada con Angular, accesible desde navegadores modernos (Google Chrome, Edge, Firefox).

Características principales:

- *Diseño basado en componentes reutilizables (formularios, tablas, modales).*
- *Navegación mediante barra lateral con módulos de Productos, Categorías, Unidades, Movimientos, Usuarios y Reportes.*
- *Validaciones visuales y mensajes de confirmación en cada acción (registro, eliminación, edición).*
- *Soporte para temas claros y oscuros.*
- *Campos con autocompletado y filtros dinámicos en tablas.*
- *Acceso restringido según rol (Administrador, Bodeguero, Auditor).*

Ejemplo de pantallas:

- *Pantalla de Login: campo de usuario y contraseña, autenticación mediante JWT.*
- *Dashboard principal: resumen de stock, últimas operaciones, alertas de bajo inventario.*
- *Pantallas CRUD: formularios para gestión de productos y categorías con validación de datos en tiempo real.*

3.1.2 Interfaces de hardware

SCAFAME no depende de hardware especializado, pero requiere un entorno mínimo para su correcto funcionamiento.

Requisitos mínimos:

- *Servidor físico o virtual con capacidad para ejecutar contenedores Docker.*
- *Conectividad de red local o VPN institucional.*
- *Equipos cliente con navegadores modernos, resolución mínima 1366x768.*

Configuración recomendada:

- *CPU: 2 núcleos o más.*
- *RAM: 4 GB mínimo (8 GB recomendado).*
- *Almacenamiento: 20 GB libres.*

El sistema puede ser implementado en servidores Linux o Windows que soporten

Docker y PostgreSQL.

3.1.3 Interfaces de software

SCAFAME interactúa con múltiples componentes y tecnologías de software que definen su ecosistema.

Descripción del software utilizado:

- *Frontend: Angular (versión 17 o superior).*
- *Backend: NestJS (framework Node.js).*
- *Base de datos: PostgreSQL 15.*
- *Contenedores: Docker Compose para orquestación de servicios.*

Propósito del interfaz:

- *Facilitar la comunicación entre frontend y backend mediante una API RESTful.*
- *Asegurar persistencia de datos mediante conexión estable con PostgreSQL.*
- *Garantizar despliegues reproducibles mediante Docker.*

Definición del interfaz (contenido y formato):

- *Protocolo: HTTPS.*
- *Formato de intercambio: JSON.*
- *Autenticación: JWT (JSON Web Token).*
- *Estructura de rutas:*
 - */api/products*
 - */api/categories*
 - */api/movements*
 - */api/reports*
 - */api/users*

3.1.4 Interfaces de comunicación

SCAFAME se comunica principalmente a través de una red local o VPN que conecta las estaciones cliente con el servidor Docker.

Detalles:

- *Protocolo de comunicación: HTTP/HTTPS.*
- *Puerto por defecto: 8080 para el backend NestJS y 5432 para PostgreSQL.*
- *Manejo de errores: respuestas HTTP estandarizadas (400, 401, 404, 500).*
- *Sincronización de datos en tiempo real mediante servicios REST y WebSockets (para actualizaciones de stock).*

- Las comunicaciones estarán cifradas mediante HTTP.

3.2 Requisitos funcionales

3.2.1 Requisito funcional 1 Login y Autenticación

Número de requisito	RF-001
Nombre de requisito	Login y Autenticación
Tipo	Requisito
Fuente del requisito	Especificación de Requisitos
Prioridad del requisito	Alta

Descripción:

- El sistema debe permitir el acceso de usuarios registrados mediante un formulario de autenticación.
- Comprobación de validez: Validar que usuario y contraseña no estén vacíos.
- Secuencia: Enviar credenciales al servidor NestJS → verificar en BD PostgreSQL → generar token JWT → redirigir al panel.
- Respuestas a errores: Si las credenciales no son válidas, devolver mensaje “Usuario o contraseña incorrectos”.
- Parámetros: Usuario, contraseña.
- Salida: Token JWT y redirección al dashboard según rol.
- Almacenamiento: Guardar hash de contraseñas y token temporal en BD.

3.2.2 Requisito funcional 2 Gestión de Usuarios

Número de requisito	RF-002
Nombre de requisito	Gestión de Usuarios
Tipo	Requisito
Fuente del requisito	Especificación de Requisitos
Prioridad del requisito	Alta

Descripción:

- El sistema debe permitir al administrador crear, editar, eliminar y consultar usuarios.
- Comprobación: Validar campos requeridos (nombre, usuario, rol, contraseña).
- Secuencia: Acceso → CRUD en tabla users → guardar cambios en BD.
- Respuestas a errores: Notificar si el nombre de usuario ya existe o si hay error de conexión.
- Parámetros: Datos de usuario (nombre, email, rol, contraseña).
- Salida: Confirmación o error visual.
- Almacenamiento: Tabla users en PostgreSQL.

3.2.3 Requisito funcional 3 Gestión de Categorías

Número de requisito	RF-003
Nombre de requisito	Gestión de Categorías
Tipo	Requisito
Fuente del requisito	Especificación de Requisitos
Prioridad del requisito	Alta

Descripción:

- Permite administrar las categorías a las que pertenecen los productos.

- Comprobación: Validar que el nombre de categoría no esté vacío o duplicado.
- Secuencia: Crear o actualizar registro → guardar en tabla categories.
- Errores: Mostrar aviso si la categoría ya existe o falta conexión.
- Parámetros: Nombre, descripción.
- Salida: Confirmación visual o alerta.
- Almacenamiento: Tabla categories.

3.2.4 Requisito funcional 4 Gestión de Productos

Número de requisito	RF-004
Nombre de requisito	Gestión de Productos
Tipo	Requisito
Fuente del requisito	Especificación de Requisitos
Prioridad del requisito	Alta

Descripción:

- El sistema debe permitir registrar productos con su categoría, unidad y cantidad inicial.
- Comprobación: Validar campos requeridos (nombre, código, unidad, categoría).
- Secuencia: Crear producto → asociar a categoría y unidad → guardar en BD.
- Errores: Mostrar alertas en caso de duplicados o datos inválidos.
- Parámetros: Código, nombre, categoría, unidad, cantidad inicial.
- Salida: Producto registrado exitosamente.
- Almacenamiento: Tabla products.

3.2.5 Requisito funcional 5 Gestión de Unidades

Número de requisito	RF-005
Nombre de requisito	Gestión de Unidades
Tipo	Requisito
Fuente del requisito	Especificación de Requisitos
Prioridad del requisito	Alta

Descripción:

- Permite definir y administrar unidades de medida utilizadas en el sistema.
- Comprobación: Validar que el nombre no esté vacío ni duplicado.
- Secuencia: Registrar o modificar unidad → guardar cambios en BD.
- Errores: Mostrar mensajes de validación.
- Parámetros: Nombre, símbolo, descripción.
- Salida: Confirmación o error.
- Almacenamiento: Tabla units.

3.2.6 Requisito funcional 6 Visualización de Inventario

Número de requisito	RF-006
Nombre de requisito	Visualización de Inventario
Tipo	Requisito
Fuente del requisito	Especificación de Requisitos
Prioridad del requisito	Alta

Descripción:

- Permite visualizar el inventario actualizado con filtros y totales.
- Comprobación: Validar existencia de datos en BD.

- Secuencia: Solicitar lista de productos → mostrar stock total.
- Errores: Si no hay datos, mostrar mensaje “No existen registros disponibles”.
- Parámetros: Categoría, rango de fechas, producto.
- Salida: Tabla de productos con stock, entradas y salidas.
- Almacenamiento: Tablas products, movements.

3.2.7 Requisito funcional 7 Realizar Ingresos

Número de requisito	RF-007
Nombre de requisito	Realizar Ingresos
Tipo	Requisito
Fuente del requisito	Especificación de Requisitos
Prioridad del requisito	Media

Descripción:

- Permite registrar ingresos de productos al inventario.
- Comprobación: Validar existencia del producto y cantidad positiva.
- Secuencia: Seleccionar producto → ingresar cantidad → actualizar stock.
- Errores: Mostrar mensajes por cantidad inválida o producto inexistente.
- Parámetros: Código del producto, cantidad, usuario responsable, fecha.
- Salida: Confirmación de registro y actualización de stock.
- Almacenamiento: Tabla movements con tipo “Ingreso”.

3.2.8 Requisito funcional 8 Solicitar Retiros

Número de requisito	RF-008
Nombre de requisito	Solicitar Retiros
Tipo	Requisito
Fuente del requisito	Especificación de Requisitos
Prioridad del requisito	Media

Descripción:

- El sistema debe permitir a los usuarios bodegueros registrar solicitudes de retiro de productos desde el inventario, quedando pendientes de aprobación por el administrador.
- Comprobación: Validar existencia del producto y cantidad disponible.
- Secuencia: Usuario selecciona producto → ingresa cantidad → genera solicitud → guarda registro en estado “Pendiente”.
- Respuestas a errores: Mostrar alerta si el producto no existe o si la cantidad supera el stock.
- Parámetros: Código del producto, cantidad solicitada, usuario, fecha.
- Salida: Confirmación de solicitud registrada.
- Almacenamiento: Tabla movements con tipo “Retiro pendiente”.

3.2.9 Requisito funcional 9 Administrar Retiros

Número de requisito	RF-009
Nombre de requisito	Solicitar Retiros
Tipo	Requisito
Fuente del requisito	Especificación de Requisitos
Prioridad del requisito	Media

Descripción:

- Permite al administrador aprobar o rechazar solicitudes de retiro de productos realizadas por los bodegueros.
- Comprobación: Validar que la solicitud esté en estado “Pendiente”.
- Secuencia: Cargar lista de solicitudes → seleccionar una → aprobar o rechazar → actualizar estado → ajustar stock si se aprueba.
- Respuestas a errores: Mostrar mensaje si la solicitud ya fue procesada o si ocurre un error de conexión.
- Parámetros: ID de solicitud, decisión (Aprobado/Rechazado), usuario aprobador.
- Salida: Mensaje de confirmación y actualización de stock.
- Almacenamiento: Tabla movements con tipo “Salida” o “Rechazado”.

3.2.10 Requisito funcional 10 Historial de Movimientos

Número de requisito	RF-010
Nombre de requisito	Solicitar Retiros
Tipo	Requisito
Fuente del requisito	Especificación de Requisitos
Prioridad del requisito	Baja

Descripción:

- El sistema debe permitir visualizar un historial completo de todos los movimientos realizados (ingresos, retiros, aprobaciones).
- Comprobación: Validar acceso de usuario con permisos de consulta.
- Secuencia: Consultar datos → aplicar filtros (fecha, tipo, usuario, producto) → mostrar resultados.
- Respuestas a errores: Si no existen registros, mostrar mensaje “No se encontraron movimientos”.
- Parámetros: Fecha inicial, fecha final, tipo de movimiento, usuario.
- Salida: Listado detallado de movimientos con columnas de tipo, fecha, producto, cantidad y usuario.
- Almacenamiento: Tabla movements.

3.2.11 Requisito funcional 11 Generación de Reportes

Número de requisito	RF-011
Nombre de requisito	Solicitar Retiros
Tipo	Requisito
Fuente del requisito	Especificación de Requisitos
Prioridad del requisito	Baja

Descripción:

- El sistema debe generar reportes de inventario y movimientos con opción de exportación a formatos PDF y Excel.
- Comprobación: Validar parámetros seleccionados por el usuario.
- Secuencia: Seleccionar tipo de reporte → definir rango de fechas → generar consulta → renderizar resultado → exportar archivo.
- Respuestas a errores: Notificar si no hay datos o falla la generación.
- Parámetros: Tipo de reporte, fecha inicial, fecha final, usuario solicitante.
- Salida: Archivo descargable (.pdf o .xlsx).
- Almacenamiento: Consulta sobre tablas products y movements.

3.3 Requisitos no funcionales

3.3.1 Requisitos de rendimiento

El sistema SCAFAME debe garantizar un rendimiento óptimo bajo condiciones de carga normales y moderadamente altas.

Especificaciones de rendimiento:

- *El tiempo promedio de respuesta del servidor no debe superar los 1,5 segundos en el 95% de las solicitudes.*
- *El sistema debe soportar al menos 5 usuarios concurrentes conectados a través de la red institucional sin degradar su desempeño.*
- *Las consultas de inventario y reportes deben generarse en menos de 2 segundos.*
- *Las operaciones CRUD sobre entidades (productos, usuarios, movimientos, categorías) deben ejecutarse en menos de 1 segundo.*
- *La arquitectura basada en NestJS + PostgreSQL deberá optimizar el uso de caché y consultas indexadas.*

Criterio de aceptación:

Durante pruebas de carga (con JMeter o similar), el sistema debe mantener una tasa de éxito $\geq 98\%$ en operaciones simultáneas.

3.3.2 Seguridad

SCAFAME debe garantizar la integridad, confidencialidad y disponibilidad de la información almacenada y transmitida.

Requisitos específicos:

- *Autenticación y autorización mediante JWT (JSON Web Tokens).*
- *Encriptación de contraseñas con bcrypt antes de su almacenamiento en la base de datos.*
- *Todas las conexiones deben realizarse sobre HTTPS utilizando certificados SSL válidos.*
- *El sistema debe mantener logs de auditoría para registrar accesos, operaciones CRUD y errores críticos.*
- *Asignación de permisos por roles:*
 - *Administrador: acceso total al sistema.*
 - *Bodeguero: movimientos e inventario.*
 - *Auditor: solo lectura de reportes y logs.*
- *Restricción de llamadas no autenticadas a la API REST (middleware de seguridad en NestJS).*
- *Copias de seguridad automáticas de la base de datos cada 24 horas.*

Criterio de aceptación:

Toda comunicación debe estar cifrada y los tokens JWT deben invalidarse tras 30 minutos de inactividad o cierre de sesión.

3.3.3 Fiabilidad

El sistema debe asegurar un funcionamiento continuo y predecible, minimizando fallos o interrupciones.

Requisitos específicos:

- *Tolerancia a errores en conexiones o caídas momentáneas del servidor.*
- *Mecanismo de reconexión automática en caso de pérdida de sesión.*
- *Gestión de excepciones a nivel de backend con control de respuestas HTTP personalizadas.*
- *Tasa máxima de error tolerable: 1 fallo por cada 1000 operaciones.*
- *Integración con herramientas de monitoreo de contenedores (Docker logs, Healthcheck).*

Criterio de aceptación:

El sistema debe mantener un MTBF (Mean Time Between Failures) mínimo de 30 días.

3.3.4 Disponibilidad

El sistema deberá estar disponible para los usuarios durante la mayor parte del horario operativo de FAME S.A.

Requisitos específicos:

- *Disponibilidad mínima del 99% mensual.*
- *Mantenimiento programado fuera del horario laboral (18h00–06h00).*
- *Los servicios deben desplegarse en contenedores Docker para garantizar una recuperación rápida en caso de falla.*
- *Se debe disponer de copias de seguridad en un almacenamiento alternativo (servidor espejo o backup externo).*
- *Los logs de disponibilidad deberán conservarse durante al menos 90 días.*

Criterio de aceptación:

En pruebas simuladas, el sistema debe restaurar su funcionamiento en menos de 5 minutos tras una caída planificada o imprevista.

3.3.5 Mantenibilidad

El sistema SCAFAME deberá diseñarse bajo principios de modularidad y separación de responsabilidades para facilitar su mantenimiento a largo plazo.

Requisitos específicos:

- *El código fuente estará estructurado en módulos independientes (usuarios, productos, movimientos, reportes), lo que permitirá la localización rápida de errores o modificaciones.*
- *El mantenimiento correctivo y evolutivo será realizado por personal*

técnico del área de desarrollo de FAME S.A. o por el equipo desarrollador (Miguel Gutiérrez y colaboradores).

- Las tareas de mantenimiento incluirán:
 - Actualización de dependencias y librerías en Angular, NestJS y PostgreSQL.
 - Monitoreo y rotación de logs en los contenedores Docker.
 - Generación de reportes semanales y mensuales de acceso al sistema.
 - Revisión de alertas de seguridad y aplicación de parches.
- El sistema debe permitir la configuración de variables de entorno (.env) sin necesidad de recompilar el código.
- Toda modificación en producción deberá realizarse previa prueba en entorno de desarrollo controlado.

Criterio de aceptación:

El mantenimiento rutinario no deberá interrumpir el servicio por más de 15 minutos y los cambios deberán documentarse en un registro de versiones (changelog).

3.3.6 Portabilidad

SCAFAME deberá ser fácilmente trasladable a distintos entornos operativos y de despliegue gracias a su arquitectura basada en contenedores.

Requisitos específicos:

- Dependencia del servidor: menor al 20% del total del sistema.
- El 100% de los servicios se ejecutarán dentro de contenedores Docker, lo que permite replicar el entorno en cualquier servidor Linux o Windows.
- Lenguaje y frameworks:
 - Frontend: Angular (TypeScript) por su portabilidad multiplataforma.
 - Backend: NestJS (Node.js) con compatibilidad en cualquier SO con soporte para Node ≥ 18 .
- Base de datos: PostgreSQL, exportable mediante archivos .sql o pg_dump.
- El sistema deberá poder desplegarse localmente, en la red de FAME S.A. o en un entorno en la nube (ej. Oracle Cloud, AWS).
- Compatibilidad de navegador: Chrome, Firefox y Edge en versiones posteriores a 2023.

Criterio de aceptación:

El sistema debe poder ejecutarse en un nuevo entorno Docker con un solo comando (docker-compose up) sin requerir cambios en el código fuente.

3.4 Otros requisitos

Requisitos culturales y organizativos:

- *La interfaz y documentación estarán completamente en idioma español.*
- *Los mensajes y reportes seguirán la terminología institucional de FAME S.A.*
- *El sistema deberá cumplir con las políticas de transparencia, control y trazabilidad interna del Ejército Ecuatoriano.*

Requisitos legales:

- *Cumplimiento de la Ley Orgánica de Protección de Datos Personales (LOPD) del Ecuador.*
- *El almacenamiento de información personal (usuarios, registros de acceso) debe cumplir con las normas de confidencialidad y seguridad digital establecidas por FAME S.A.*
- *El software no debe infringir derechos de propiedad intelectual ni utilizar componentes sin licencia adecuada.*

Criterio de aceptación:

Todo el sistema y sus librerías deberán tener licencias libres (MIT, Apache 2.0 o equivalentes) y cumplir con las regulaciones nacionales vigentes.

4 Apéndices

A.1 Modelo de Datos:

Incluye los diagramas de entidad-relación que describen la estructura de las tablas users, products, categories, units y movements.

A.2 Diagramas de Arquitectura:

Representación del flujo general entre los componentes Angular → NestJS → PostgreSQL, dentro de la red Docker.

A.3 Manual de Instalación y Configuración:

Guía paso a paso para desplegar el sistema en un entorno nuevo:

1. *Clonar el repositorio.*
2. *Configurar archivo .env con credenciales de base de datos y puertos.*
3. *Ejecutar docker-compose up -d.*
4. *Acceder desde navegador a <http://localhost:8080>.*

A.4 Evidencias de Pruebas:

Capturas de las pruebas unitarias, de integración y validación funcional realizadas antes de la entrega.

A.5 Control de Versiones:

Listado de cambios, correcciones y nuevas funcionalidades agregadas en cada iteración.